Day 39

Introduction to Object Oriented Programming:

Object-Oriented Programming (OOP) is a way of writing code where we group related data and functions into a single unit called an object. It helps us structure programs so they are easy to understand, reuse, and maintain.

In JavaScript, OOP is based on objects. Each object stores:

- **Properties** → data
- **Methods** → functions that belong to the object

Using OOP, we model real-world things as code.
Example: A *car* has properties like color, speed, and methods like start() or stop().

Why Use OOP?

- Organizes code in a clean way
- Makes code reusable
- Helps create multiple objects easily
- Reduces errors by grouping related things together

Four Pillars of OOP in JavaScript:

1. **Encapsulation**
   - Wrap data + functions into one unit (object)
2. **Abstraction**
   - Hide complexity and show only what is needed
3. **Inheritance**
   - One object can get properties and methods from another
4. **Polymorphism**
   - Same method name behaves differently in different objects

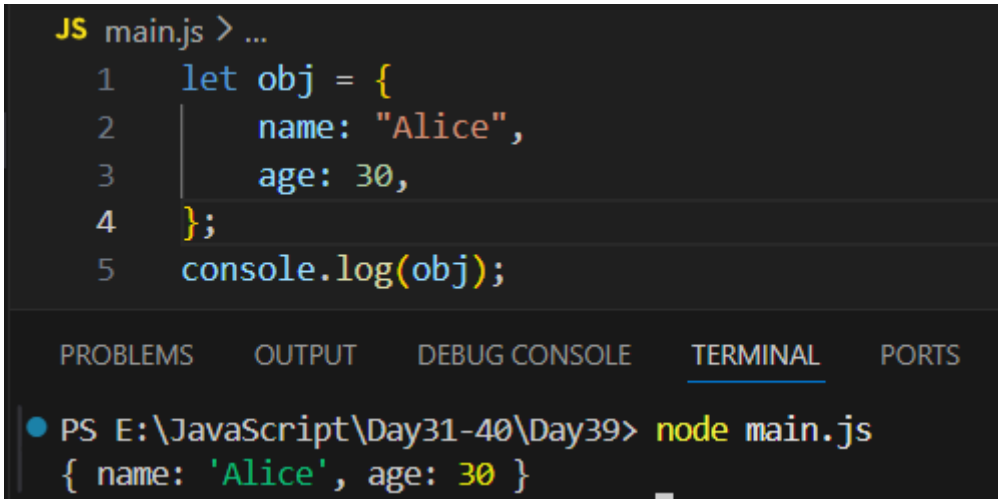
Protypes and __proto__ in JavaScript:

In JavaScript, every object has a hidden link to another object called its prototype, and this prototype is where JavaScript looks for properties or methods if they are not found on the original object.

Think of it like a backup or a parent object. The __proto__ property (pronounced “dunder proto”) is simply a pointer that shows which prototype your object is connected to. When you access obj.someFunction, JavaScript first checks inside obj; if it can’t find it, it climbs up the chain using obj.__proto__, then the prototype’s prototype, and so on—this is called the prototype chain.

Functions in JS also have a special property called prototype, which is used when creating objects with constructors or classes; this prototype becomes the __proto__ of all objects created by that constructor. This system allows JavaScript to share methods efficiently, avoid duplicates, and support OOP-style behaviour.

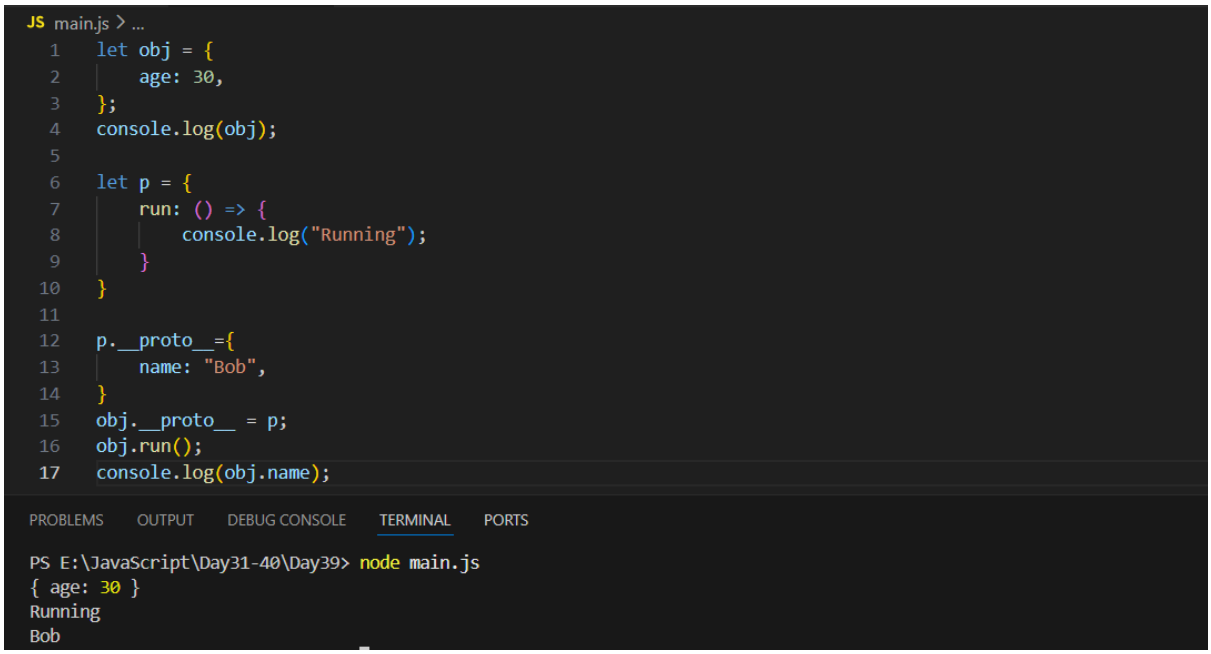
Example: creating a basic object in the JS



Now, let's create a method:

So, will this method can be used by our object obj? No, to let it do so:

Now, we can create prototype of a prototype as well:



--The End--